

v1.3 — Database & Data Model Specification

Absolutely. v1.3 is the point where we turn the backend domain model into a concrete relational data model. I'll keep the schema extensible and make tenant isolation, auditability, document versioning, Icelandic naming, and records-management requirements explicit.

ILCMS — Icelandic Legal Case Management System

Technical Specification v1.3 — Database & Data Model Specification

Document version: 1.3

Status: Engineering Specification

Target market: Iceland

Primary database: PostgreSQL

ORM: SQLAlchemy 2.x

Migration framework: Alembic

Related specifications: v1.1 DevOps & Kubernetes; v1.2 Backend Implementation

1. Purpose

This specification defines the PostgreSQL database architecture and logical data model for ILCMS.

The database shall provide the authoritative transactional store for:

- organizations and tenants
- users and access control
- cases
- parties
- case/party relationships
- classifications
- documents
- document versions
- tasks
- deadlines
- notes and communications
- notifications

- audit events
- background jobs
- integrations
- retention and archival metadata

The database shall be designed for:

- strong data integrity
- multi-tenant isolation
- auditability
- high availability
- controlled evolution
- efficient search
- legal-document management
- future scalability

2. Database Principles

The database design shall follow these principles:

1. PostgreSQL is the authoritative transactional data store.
2. Every tenant-owned entity shall be associated with an organization.
3. Foreign keys shall enforce relationships wherever practical.
4. Business-critical constraints shall be enforced in the database as well as the application.
5. UUIDs shall be used for externally exposed entity identifiers.
6. Timestamps shall be timezone-aware.
7. Existing document versions shall be immutable.
8. Audit events shall be append-only.
9. Derived data shall be distinguishable from authoritative data.
10. Binary documents shall normally reside in object storage, not PostgreSQL.
11. Database migrations shall be managed exclusively through Alembic.
12. Destructive schema changes shall follow expand/contract migration practices.

13. Indexes shall be designed around actual query patterns.
14. Sensitive data shall be encrypted at rest through infrastructure/database mechanisms.
15. PostgreSQL Row-Level Security may be used as an additional tenant-isolation control.

3. PostgreSQL Version

The production PostgreSQL version shall be a currently supported major release.

The exact version shall be pinned by the deployment platform.

The development, test, staging and production environments shall use compatible PostgreSQL major versions.

4. Database Namespaces

The initial implementation should use a dedicated application schema.

Example:

```
CREATE SCHEMA ilcms;
```

Potential future schemas:

```
ilcms
```

```
audit
```

```
integration
```

```
reporting
```

For the MVP, keeping the core transactional model in one schema is recommended.

5. Identifier Strategy

All primary identifiers shall use UUID.

Recommended PostgreSQL implementation:

```
id UUID PRIMARY KEY
```

UUID generation should preferably occur using a supported PostgreSQL UUID generation mechanism or within the application.

Identifiers exposed through the API shall not reveal database sequence information.

6. Human-Readable Case Numbers

A case shall have both:

`id`

`case_number`

Example:

`id:`

`550e8400-e29b-41d4-a716-446655440000`

`case_number:`

`2026-00157`

`id` is the technical identifier.

`case_number` is the human-facing identifier.

Case numbers shall be unique within an organization.

7. Common Timestamp Fields

Most mutable entities should contain:

`created_at`

`updated_at`

Example:

`created_at TIMESTAMPTZ NOT NULL DEFAULT now(),`

`updated_at TIMESTAMPTZ NOT NULL DEFAULT now()`

Where relevant:

`deleted_at`

`archived_at`

`closed_at`

`completed_at`

Soft deletion shall not be automatically applied to every table.

It shall be used only where the business requirements justify it.

8. Organizations

The `organizations` table represents a legal/customer tenant.

Conceptual schema:

`organizations`

`id`

name
display_name
national_id
organization_type
status
default_locale
default_timezone
created_at
updated_at
Suggested organization types:

LAW_FIRM
LEGAL_DEPARTMENT
CORPORATION
PUBLIC_BODY
OTHER

9. Organization Constraints

Example:

```
id UUID PRIMARY KEY,  
name VARCHAR(255) NOT NULL,  
status VARCHAR(32) NOT NULL,  
default_locale VARCHAR(16) NOT NULL DEFAULT 'is-IS',  
default_timezone VARCHAR(64) NOT NULL DEFAULT 'Atlantic/  
Reykjavik'
```

If `national_id` is stored, it shall have appropriate uniqueness rules and validation.

The application shall not assume that every organization has an Icelandic national ID.

10. Users

Users represent authenticated system users.

users

id
external_subject
email
display_name
given_name
family_name
active

locale
timezone
created_at
updated_at
external_subject identifies the user in the configured identity provider.

Example:

OIDC issuer + subject

The backend shall not use email address as the immutable user identifier.

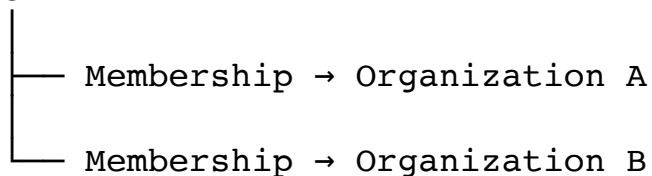
11. User Organization Membership

A user may belong to one or more organizations.

organization_memberships

id
organization_id
user_id
status
created_at
updated_at
Relationship:

User



This permits future support for users working with multiple legal organizations.

12. Roles

Roles shall be stored separately from users.

roles

id
organization_id nullable
name
description
system_role
created_at

System roles may be predefined.

Organization-specific roles may be created by administrators.

13. Permissions

permissions

id

code

description

Examples:

case.read

case.create

case.update

case.assign

case.close

document.read

document.upload

document.download

document.delete

audit.read

user.manage

organization.manage

Permission codes shall be unique.

14. Role-Permission Relationship

role_permissions

role_id

permission_id

Composite primary key:

PRIMARY KEY (role_id, permission_id)

15. User-Role Relationship

membership_roles

membership_id
role_id

A role should normally be assigned within the context of an organization membership.

16. Cases

The **cases** table is the central transactional entity.

```
cases
-----
id
organization_id
case_number
title
description
case_type
classification_id
status
priority
responsible_user_id
opened_at
closed_at
archived_at
created_by
created_at
updated_at
```

17. Case Constraints

At minimum:

PRIMARY KEY (id)

UNIQUE (organization_id, case_number)

FOREIGN KEY (organization_id)
REFERENCES organizations(id)

FOREIGN KEY (created_by)
REFERENCES users(id)

Case title shall be mandatory.

Case number shall be unique within the organization.

18. Case Status

The initial controlled vocabulary:

DRAFT

OPEN

ACTIVE

ON_HOLD

CLOSED

ARCHIVED

The application shall enforce permitted state transitions.

The database may additionally enforce valid status values through an enum or check constraint.

A lookup-table approach is preferred if statuses are intended to become configurable.

19. Case Priority

Initial values:

LOW

NORMAL

HIGH

URGENT

Priority is an operational attribute and shall not automatically imply legal urgency.

20. Case Types

Case types should be configurable.

Examples:

CIVIL

CRIMINAL

EMPLOYMENT

CORPORATE

CONTRACT

PROPERTY

FAMILY

ADMINISTRATIVE

OTHER

The initial installation shall not assume that these categories are legally exhaustive.

21. Case Classification

The classification system shall be hierarchical.

classifications

id

organization_id

parent_id

code

name

description

active

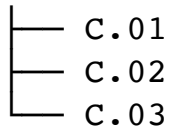
sort_order

created_at

updated_at

Example:

C



22. Classification Constraints

Recommended:

UNIQUE (organization_id, code)

parent_id shall reference the same classification table.

The application shall prevent cycles in the hierarchy.

23. Parties

A party is a reusable entity that may participate in multiple cases.

parties

id

organization_id

party_type

display_name

given_name

family_name

full_legal_name
national_id
organization_number
email
phone
address
notes
created_at
updated_at
Party types:

PERSON
ORGANIZATION
AUTHORITY
COURT
OTHER

24. Icelandic Name Model

The model shall not require a traditional first-name/last-name combination.

For a natural person:

given_name
family_name
full_legal_name
display_name
may be populated independently.

The system must support names such as:

Jón Þórðarson
Guðrún Ólafsdóttir
Anna María Jónsdóttir
without imposing assumptions about surname inheritance or structure.

25. Party-to-Case Relationship

A party may have different roles in different cases.

Therefore the relationship shall be represented by a junction table.

case_parties

id
case_id

party_id
role
description
is_primary
created_at

Examples of roles:

CLIENT
OPPOSING_PARTY
CLAIMANT
DEFENDANT
WITNESS
COURT
AUTHORITY
COUNSEL
CONTACT
OTHER

26. Case Party Constraints

Recommended:

UNIQUE (case_id, party_id, role)

This prevents duplicate assignment of the same party to the same role.

27. Party Contacts

A party may have multiple contact methods.

A separate table is recommended:

party_contacts

id
party_id
contact_type
value
is_primary
verified_at
created_at
updated_at
Types:

EMAIL
PHONE

MOBILE

OTHER

This is preferable to limiting a party to one email address or phone number.

28. Addresses

Addresses should be normalized separately if multiple addresses are required.

party_addresses

id

party_id

address_type

street

postal_code

city

country_code

is_primary

created_at

updated_at

Country codes should follow a defined standard such as ISO 3166-1 alpha-2.

29. Case Team

A case may have several users assigned to it.

case_users

id

case_id

user_id

role

is_primary

created_at

Possible roles:

RESPONSIBLE_LAWYER

ASSISTANT

PARALEGAL

REVIEWER

OBSERVER

OTHER

This is preferable to having only a single `responsible_user_id`.

The `responsible_user_id` field may remain as a convenience field for the primary responsible person.

30. Documents

`documents`

`id`
`organization_id`
`case_id`
`title`
`document_type`
`classification_id`
`status`
`sender_party_id`
`recipient_party_id`
`document_date`
`received_at`
`current_version_id`
`created_by`
`created_at`
`updated_at`

A document belongs to an organization and normally to a case.

Future requirements may permit documents outside a case.

31. Document Status

Initial states:

`UPLOAD_PENDING`
`UPLOADED`
`SCANNING`
`PROCESSING`
`READY`
`FAILED`
`ARCHIVED`

Only valid transitions shall be permitted.

32. Document Types

Document types shall be configurable.

Examples:

PLEADING
COURT_DECISION
CONTRACT
LETTER
EMAIL
REPORT
MEMO
EVIDENCE
INVOICE
OTHER

The classification system and document type system shall remain separate.

33. Document Versions

```
document_versions
-----
id
document_id
version_number
object_storage_key
checksum_sha256
mime_type
file_size
original_filename
storage_bucket
created_by
created_at
Each version shall be immutable.
```

34. Document Version Constraints

Recommended:

```
UNIQUE (document_id, version_number)
UNIQUE (document_id, checksum_sha256)
```

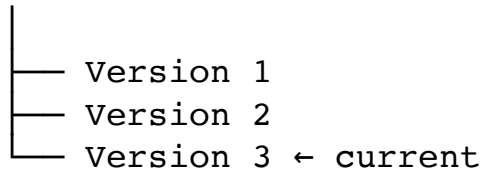
The second constraint may be relaxed if identical files are legitimately uploaded as separate versions.

35. Current Document Version

`documents.current_version_id` shall reference the current version.

This allows:

Document



Previous versions remain available according to authorization and retention policy.

36. Document Metadata

The system shall permit extensible metadata.

A controlled relational model shall be used for important searchable fields.

Potential additional metadata:

`language`
`source`
`recipient`
`sender`
`document_reference`
`external_reference`
`page_count`
`ocr_status`

Arbitrary metadata should not replace relational columns for frequently queried business-critical information.

37. Object Storage References

The database shall store references to object storage rather than the binary content itself.

Example:

`storage_bucket`
`object_storage_key`
`checksum_sha256`

The database shall not store presigned URLs permanently.

URLs shall be generated dynamically.

38. Document Processing

Processing state may be tracked with:

`document_processing_jobs`

`id`

`document_version_id`

`job_type`

`status`

`attempt_count`

`started_at`

`completed_at`

`error_code`

`created_at`

`updated_at`

Job types:

`VIRUS_SCAN`

`OCR`

`TEXT_EXTRACTION`

`INDEXING`

`THUMBNAIL`

39. Extracted Text

Extracted text shall be treated as derived information.

Recommended:

`document_text`

`id`

`document_version_id`

`text_content`

`language`

`extraction_method`

`created_at`

For very large documents, the implementation may use external search storage rather than keeping all extracted text in PostgreSQL.

40. Document Hashing

Every document version should contain:

`checksum_sha256`

This provides integrity verification.

Example:

SHA-256(document bytes)

The checksum shall be calculated by trusted backend processing, not blindly accepted from the client.

41. Tasks

tasks

id

organization_id

case_id

title

description

status

priority

assigned_to

created_by

due_at

completed_at

created_at

updated_at

A task may optionally be associated with a document.

42. Task Status

Initial values:

OPEN

IN_PROGRESS

BLOCKED

COMPLETED

CANCELLED

43. Deadlines

Legal/business deadlines shall have their own entity.

deadlines

id

organization_id

case_id
title
description
deadline_type
due_at
status
source
completed_at
created_by
created_at
updated_at

44. Deadline Type

Initial values:

LEGAL
COURT
CONTRACTUAL
INTERNAL
REMINDER
OTHER

The system shall not infer that a deadline is legally binding merely because it has type **LEGAL**.

The user or configured workflow must provide the authoritative source.

45. Deadline Source

A deadline should optionally reference its source.

Example:

source_type
source_document_id
source_description

This permits:

Deadline

↓

Court document

↓

Specific date

without requiring the database to interpret the legal validity of the deadline.

46. Notes

Internal case notes should be stored separately from formal documents.

case_notes

id

case_id

author_id

content

visibility

created_at

updated_at

Visibility might include:

CASE_TEAM

ORGANIZATION

PRIVATE

The exact access rules shall be implemented by authorization services.

47. Communications

Where communications are managed by ILCMS:

communications

id

organization_id

case_id

communication_type

direction

subject

sent_at

received_at

created_by

created_at

Examples:

EMAIL

LETTER

PHONE

MEETING

COURT_COMMUNICATION

OTHER

Communication content should not be duplicated unnecessarily if the authoritative content exists as a document.

48. Notifications

```
notifications
-----
id
user_id
organization_id
type
title
message
resource_type
resource_id
read_at
created_at
```

Notifications are transient user-facing records and should not be confused with audit events.

49. Audit Events

Audit events shall be stored separately from ordinary business tables.

```
audit_events
-----
id
organization_id
actor_user_id
event_type
resource_type
resource_id
request_id
timestamp
ip_address
user_agent
metadata
```

50. Audit Event Immutability

The application shall expose no normal update/delete operation for audit events.

Database permissions should additionally prevent ordinary application users from modifying audit records.

Where practical, the audit schema should be isolated with stricter database permissions.

51. Audit Metadata

metadata may use PostgreSQL JSONB.

Example:

```
{
  "old_status": "OPEN",
  "new_status": "CLOSED"
}
```

Sensitive values shall not be included merely for convenience.

52. Audit Indexes

Recommended indexes:

```
CREATE INDEX idx_audit_resource
ON audit_events(resource_type, resource_id);
```

```
CREATE INDEX idx_audit_actor
ON audit_events(actor_user_id);
```

```
CREATE INDEX idx_audit_timestamp
ON audit_events(timestamp);
```

```
CREATE INDEX idx_audit_organization_timestamp
ON audit_events(organization_id, timestamp);
```

53. Audit Event Examples

The backend shall generate events for operations such as:

```
CASE_CREATED
CASE_UPDATED
CASE_ASSIGNED
CASE_CLOSED
```

```
DOCUMENT_UPLOADED
DOCUMENT_DOWNLOADED
DOCUMENT_VERSION_CREATED
```

PARTY_CREATED
PARTY_UPDATED

TASK_CREATED
TASK_COMPLETED

USER_ROLE_CHANGED
PERMISSION_CHANGED

DATA_EXPORTED

The final event catalog shall be maintained as part of the backend codebase.

54. Authentication Events

Authentication-related events may be stored in the audit system or dedicated security-event storage.

Examples:

LOGIN_SUCCESS
LOGIN_FAILURE
LOGOUT
TOKEN_REJECTED
MFA_EVENT

Authentication events should be correlated with identity-provider logs where possible.

55. Background Jobs

```
jobs
----
id
organization_id
job_type
resource_type
resource_id
status
attempt_count
max_attempts
scheduled_at
started_at
completed_at
last_error
created_at
updated_at
```

Jobs shall not contain large document payloads.

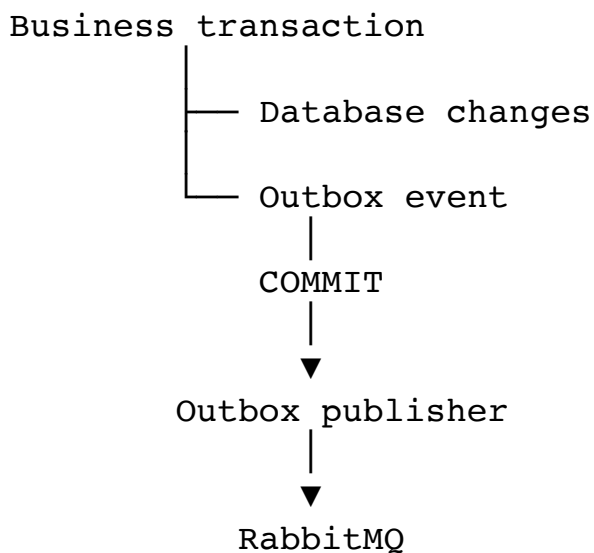
56. Outbox

A transactional outbox table shall be used where reliable event delivery is required.

```
outbox_events
-----
id
organization_id
event_type
aggregate_type
aggregate_id
payload
status
attempt_count
available_at
published_at
created_at
payload should be JSONB.
```

57. Outbox Processing

The lifecycle:



The outbox publisher shall be retryable.

58. Retention Metadata

Retention requirements should be represented as metadata rather than hard-coded destruction dates.

Potential model:

```
retention_policies
-----
id
organization_id
name
resource_type
retention_period
action
active
created_at
updated_at
Possible actions:
```

```
REVIEW
ARCHIVE
DESTROY
```

59. Retention Assignment

Resources may reference an applicable policy.

Example:

```
cases.retention_policy_id
documents.retention_policy_id
Alternatively, retention may be derived from classification/type.
```

The chosen approach must ensure that policy changes do not silently rewrite historical decisions.

60. Archival Metadata

Where archival workflows are implemented:

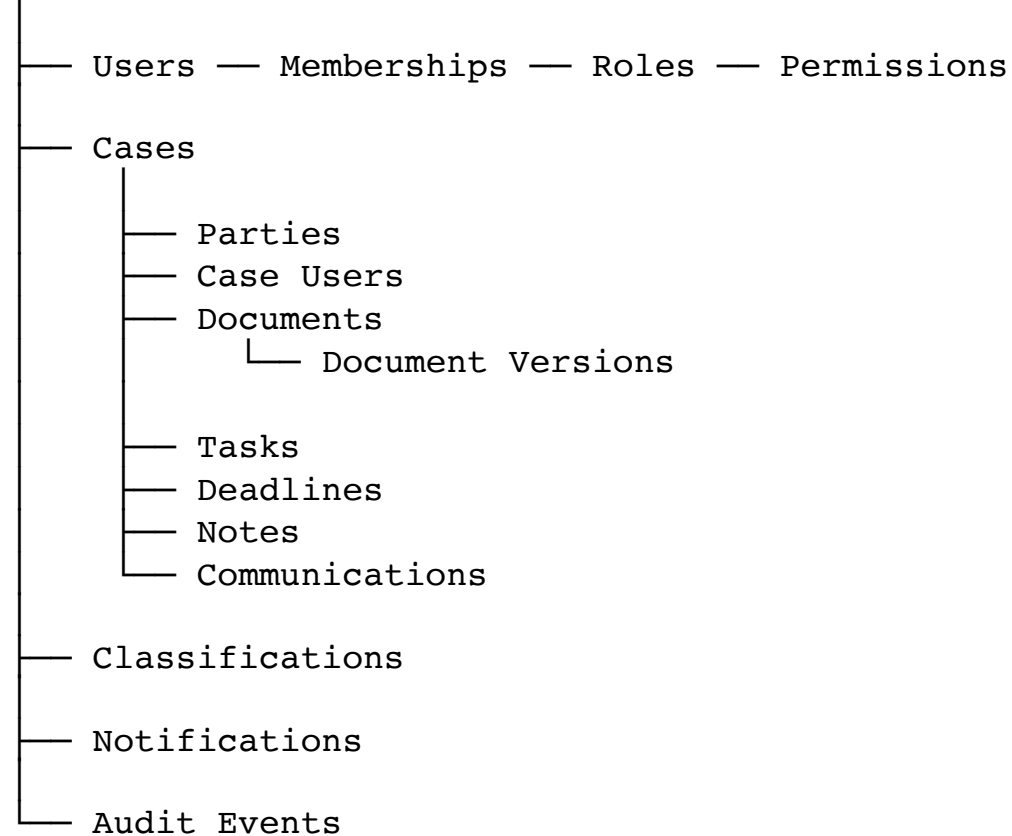
```
archive_records
-----
id
resource_type
resource_id
archive_status
archived_at
archive_reference
created_at
```

The archival record should not replace the authoritative business object.

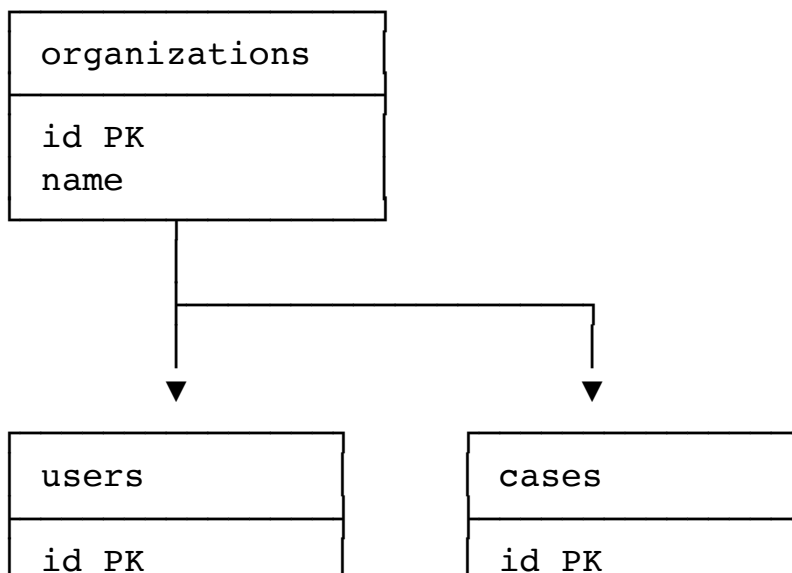
61. Database Relationships

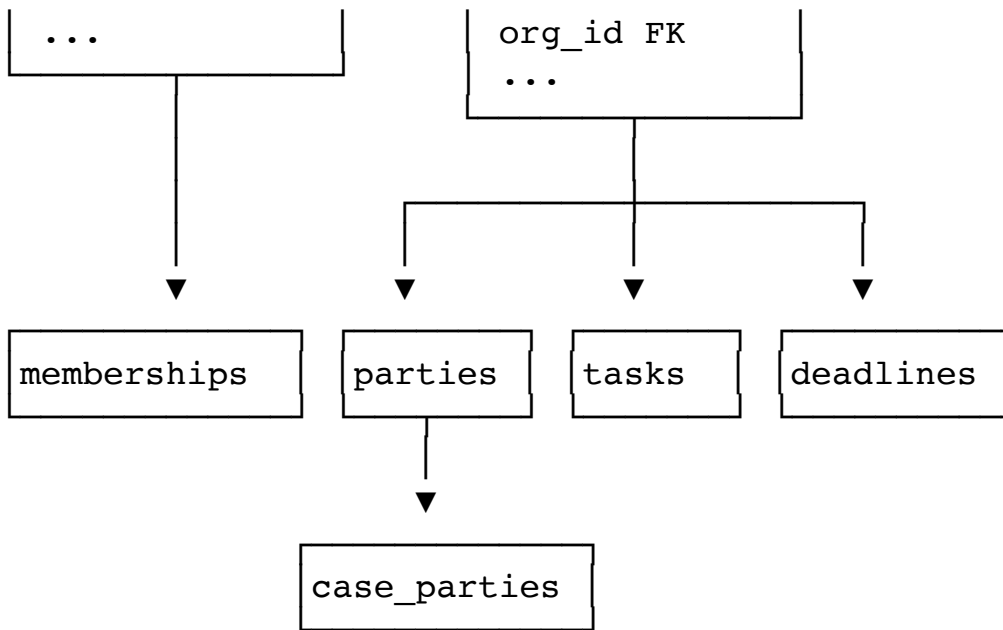
High-level relationship model:

Organization



62. Simplified ER Model





```

cases
  |
  v
documents
  |
  v
document_versions
  
```

63. Foreign-Key Strategy

Foreign keys shall be used for authoritative relationships.

Examples:

```

cases.organization_id
    → organizations.id
  
```

```

documents.case_id
    → cases.id
  
```

```

document_versions.document_id
    → documents.id
  
```

```

case_parties.case_id
    → cases.id
  
```

```

case_parties.party_id
    → parties.id
  
```

Foreign-key actions shall be explicitly selected.

Unrestricted ON DELETE CASCADE shall be avoided for legally significant data.

64. Deletion Strategy

For important legal records, deletion should normally not cascade automatically.

Example:

Organization

↓

Case

↓

Document

Deleting the organization must not accidentally delete all legal records.

Destruction shall be an explicit, controlled workflow.

65. Soft Deletion

Where soft deletion is used:

`deleted_at TIMESTAMPTZ NULL`

Queries must explicitly exclude deleted records.

For legally significant records, soft deletion alone shall not constitute a destruction mechanism.

66. Indexing Strategy

Indexes shall be created for:

- foreign keys used in joins
- organization/tenant filtering
- case number
- status
- assignment
- document relationships
- deadlines
- audit events

- common search fields

Example:

```
CREATE INDEX idx_cases_org_status  
ON cases(organization_id, status);
```

```
CREATE INDEX idx_cases_org_assignee  
ON cases(organization_id, responsible_user_id);
```

```
CREATE INDEX idx_documents_case  
ON documents(case_id);
```

67. Case Search Index

PostgreSQL full-text search may initially use a generated `tsvector`.

Example conceptual field:

```
search_vector TSVECTOR
```

It may combine:

```
case title
```

```
case description
```

```
party names
```

```
document metadata
```

The architecture must allow migration to a dedicated search platform later.

68. Icelandic Text Search

The database/search implementation shall explicitly test Icelandic text.

Test cases shall include:

```
Þjóðskjalasafn
```

```
Hæstiréttur
```

```
Héraðsdómur
```

```
Ólafsdóttir
```

```
Þórðarson
```

```
Ævar
```

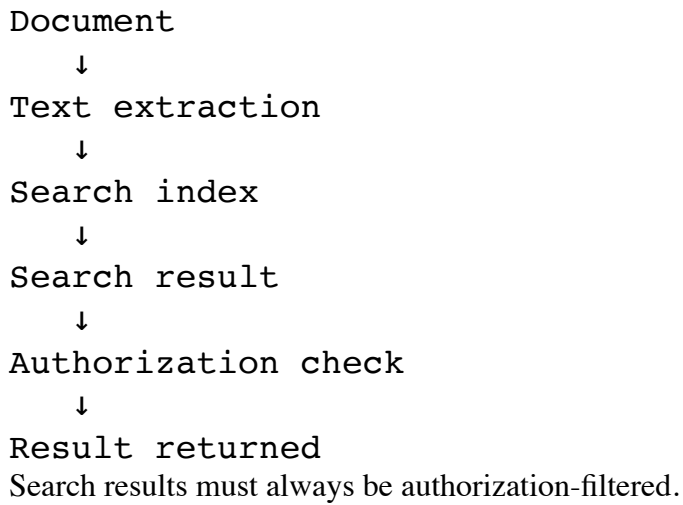
```
Guðrún
```

The search implementation must not silently remove or corrupt Icelandic characters.

69. Full-Text Document Search

Large document text shall preferably be indexed separately from the primary case tables.

Conceptual flow:



70. JSONB Usage

JSONB is appropriate for:

- flexible metadata
- audit metadata
- integration payloads
- event payloads

JSONB shall not be used as a replacement for relational modeling of core business entities.

For example, this is discouraged:

```
cases.metadata = {  
  "title": "...",  
  "status": "...",  
  "client": "..."  
}
```

Core fields shall remain typed relational columns.

71. Database-Level Tenant Isolation

The preferred defense-in-depth architecture is:

Application authorization
+

Repository organization filtering

+

PostgreSQL constraints

+

Optional Row-Level Security

Row-Level Security should be evaluated for production deployment.

72. PostgreSQL Row-Level Security

A future production configuration may use:

```
ALTER TABLE cases ENABLE ROW LEVEL SECURITY;
```

The database session would receive the authenticated organization context.

Policies could then restrict:

```
organization_id = current_setting('app.organization_id')
```

The exact mechanism must be implemented carefully to prevent context spoofing.

RLS shall supplement, not replace, application authorization.

73. Encryption

Encryption at rest shall be primarily provided by the infrastructure/database platform.

Sensitive fields that require application-level encryption should be identified explicitly.

The system should avoid unnecessary application-level encryption because it complicates:

- search
- indexing
- migrations
- key rotation
- recovery

74. Personally Identifiable Information

Potentially sensitive fields include:

`national_id`

`email`

`phone`

address
party notes
case descriptions
document metadata

The data model shall make these fields identifiable for privacy-impact analysis.

75. National Identification Numbers

If Icelandic kennitala or equivalent identifiers are stored, they shall:

- have strict access control
- be validated
- not be exposed unnecessarily in API responses
- not appear in ordinary logs
- be protected according to applicable privacy/security requirements

The database should not assume every party has such an identifier.

76. Referential Integrity

The following shall generally be impossible:

Document referencing nonexistent Case
Task referencing nonexistent Case
Case Party referencing nonexistent Party
Document Version referencing nonexistent Document
Foreign-key constraints shall enforce these relationships.

77. Uniqueness

Examples:

organization + case_number
organization + classification.code
document + version_number
role + permission
user + organization membership

Uniqueness constraints shall be defined at database level wherever the rule is authoritative.

78. Check Constraints

Database constraints should protect simple invariants.

Example:

```
CHECK (file_size >= 0)
and:
```

```
CHECK (version_number > 0)
```

More complex workflow rules should remain in the domain/service layer.

79. Optimistic Concurrency

Important mutable entities may include:

```
version INTEGER NOT NULL DEFAULT 1
```

Updates can then require:

```
WHERE id = :id
AND version = :expected_version
```

Successful updates increment the version.

This helps prevent accidental overwriting of another user's changes.

80. Audit and Transactions

Business changes and their audit records shall normally be committed within the same database transaction.

Example:

```
BEGIN
```

```
UPDATE cases
SET status = 'CLOSED'
```

```
INSERT INTO audit_events (...)
```

```
COMMIT
```

If the transaction fails, neither change should be committed.

81. Outbox and Audit Ordering

Where an operation produces both an audit event and asynchronous business event:

```
BEGIN
    business change
    audit event
    outbox event
```

```
COMMIT
```

This ensures that the database remains the source of truth for the operation.

82. Database Connection Management

The application shall use a connection pool.

Pool size shall be configured according to:

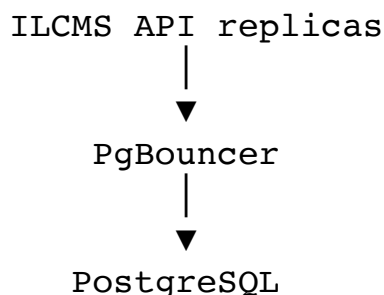
- number of API replicas
- worker replicas
- PostgreSQL capacity
- expected concurrency

Connection exhaustion shall be monitored.

83. PgBouncer

Where Kubernetes scaling produces many application connections, PgBouncer should be used.

Architecture:



The exact pooling mode shall be validated against SQLAlchemy and transaction requirements.

84. Database Migrations

All schema changes shall use Alembic.

Example:

```
alembic revision --autogenerate -m "create cases"
alembic upgrade head
```

Generated migrations shall always be reviewed manually.

Autogenerated migrations shall not be blindly deployed.

85. Migration Rules

A migration shall:

- have a descriptive name
- be deterministic
- be tested against a realistic database
- avoid unnecessary downtime
- document destructive changes
- be compatible with the deployment sequence

86. Seed Data

Reference data required for application operation shall be initialized through controlled migrations or explicit seed mechanisms.

Examples:

System roles

System permissions

Default document types

Default case statuses

Default task statuses

Tenant-specific configuration shall not be hard-coded into migrations unless intentional.

87. Backup Compatibility

The database design shall support:

- full backups
- point-in-time recovery
- WAL archiving

- replica creation
- disaster recovery

No schema design decision should depend on an unrecoverable external state.

Object-storage references must also be included in the overall backup/DR design.

88. Database Monitoring

Production monitoring should include:

Connection count
Connection saturation
Query latency
Transaction rate
Deadlocks
Lock waits
Replication lag
Disk usage
WAL generation
Cache hit ratio
Slow queries
Autovacuum health

89. Partitioning

Partitioning shall not be introduced prematurely.

Potential candidates at larger scale include:

`audit_events`
because audit records can become very large and are naturally time-oriented.

Partitioning should be introduced based on actual production volume.

90. Audit Data Growth

Audit records should be expected to grow significantly faster than some transactional tables.

The architecture shall therefore allow:

partitioning
archival
WORM storage

separate audit database
without changing the public API.

91. Data Import

The backend should eventually support controlled data import.

Import jobs should:

- validate records
- report errors
- be idempotent where possible
- generate audit events
- never bypass authorization
- support dry-run validation

92. Data Export

Exported data shall be treated as sensitive.

An export record may contain:

id
organization_id
requested_by
export_type
scope
status
object_storage_key
created_at
completed_at
Exports should expire according to configured policy.

93. Reporting

Reporting queries should not unnecessarily burden transactional workloads.

At scale, the system may use:

read replica

reporting database
materialized views
data warehouse

The initial MVP should remain simple.

94. Database Testing

Automated tests shall verify:

Schema

- migrations
- constraints
- indexes

Integrity

- foreign keys
- uniqueness
- valid relationships

Security

- tenant isolation
- RLS if enabled
- unauthorized access

Transactions

- rollback
- audit consistency
- outbox consistency

95. Example Case Schema

Conceptual SQL:

```
CREATE TABLE ilcms.cases (  
    id UUID PRIMARY KEY,
```

```

organization_id UUID NOT NULL
    REFERENCES ilcms.organizations(id),
case_number VARCHAR(64) NOT NULL,
title VARCHAR(500) NOT NULL,
description TEXT,
case_type VARCHAR(64),
classification_id UUID
    REFERENCES ilcms.classifications(id),
status VARCHAR(32) NOT NULL,
priority VARCHAR(32) NOT NULL,
responsible_user_id UUID
    REFERENCES ilcms.users(id),
opened_at TIMESTAMPTZ,
closed_at TIMESTAMPTZ,
archived_at TIMESTAMPTZ,
created_by UUID NOT NULL
    REFERENCES ilcms.users(id),
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),

CONSTRAINT uq_cases_org_number
    UNIQUE (organization_id, case_number)
);

```

This is representative rather than the final migration.

96. Example Document Version Schema

```

CREATE TABLE ilcms.document_versions (
    id UUID PRIMARY KEY,
    document_id UUID NOT NULL
        REFERENCES ilcms.documents(id),
    version_number INTEGER NOT NULL,
    object_storage_key TEXT NOT NULL,
    checksum_sha256 CHAR(64) NOT NULL,
    mime_type VARCHAR(255) NOT NULL,
    file_size BIGINT NOT NULL,
    original_filename TEXT NOT NULL,
    created_by UUID NOT NULL
        REFERENCES ilcms.users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

CONSTRAINT uq_document_version
    UNIQUE (document_id, version_number),

```

```

        CONSTRAINT chk_document_file_size
            CHECK (file_size >= 0),

        CONSTRAINT chk_document_version
            CHECK (version_number > 0)
    );

```

97. Example Audit Schema

```

CREATE TABLE ilcms.audit_events (
    id UUID PRIMARY KEY,
    organization_id UUID
        REFERENCES ilcms.organizations(id),
    actor_user_id UUID
        REFERENCES ilcms.users(id),
    event_type VARCHAR(128) NOT NULL,
    resource_type VARCHAR(128),
    resource_id UUID,
    request_id VARCHAR(128),
    timestamp TIMESTAMPTZ NOT NULL DEFAULT now(),
    ip_address INET,
    user_agent TEXT,
    metadata JSONB
);

```

The final implementation should consider whether audit records require stronger database-level isolation.

98. Complete Initial Entity Inventory

The initial database should contain approximately the following logical entities:

```

organizations
users
organization_memberships
roles
permissions
role_permissions
membership_roles

cases
classifications
case_users
case_parties

```

parties
party_contacts
party_addresses

documents
document_versions
document_processing_jobs
document_text

tasks
deadlines
case_notes
communications

notifications

audit_events
security_events

jobs
outbox_events

retention_policies
archive_records
exports

Not every entity must be implemented in the first MVP migration.

99. MVP Database Scope

The first production-capable database should prioritize:

organizations
users
organization_memberships

roles
permissions
role_permissions
membership_roles

cases
classifications
case_users
parties

case_parties

documents

document_versions

document_processing_jobs

tasks

deadlines

audit_events

outbox_events

The remaining tables can be introduced incrementally.

100. Initial Migration Sequence

Recommended migration order:

001_extensions_and_schema

002_organizations

003_users

004_memberships

005_roles_permissions

006_classifications

007_cases

008_parties

009_case_parties

010_case_users

011_documents

012_document_versions

013_document_processing

014_tasks

015_deadlines

016_audit

017_outbox

018_indexes_and_constraints

Actual migration numbering may differ.

101. Data Model Security Rules

The database implementation shall ensure:

- [] Tenant ID exists on tenant-owned entities
- [] Foreign keys are enforced

- [] Case numbers are tenant-unique
- [] Document versions are immutable
- [] Audit records are append-only
- [] National IDs receive special protection
- [] Sensitive columns are not logged
- [] No credentials are stored in business tables
- [] Object storage keys do not expose sensitive information
- [] Search data cannot bypass authorization

102. Data Model and Icelandic Records Management

The data model is intentionally capable of representing concepts relevant to Icelandic records management, including:

- case registration
- classification
- case/document association
- document dates
- sender/recipient
- case identifiers
- archival metadata
- retention policies

The model shall remain configurable so that organizations can implement their applicable records-management policies without requiring changes to the core database schema.

103. Data Model and Court Documents

The document model is designed to support court-related documents through:

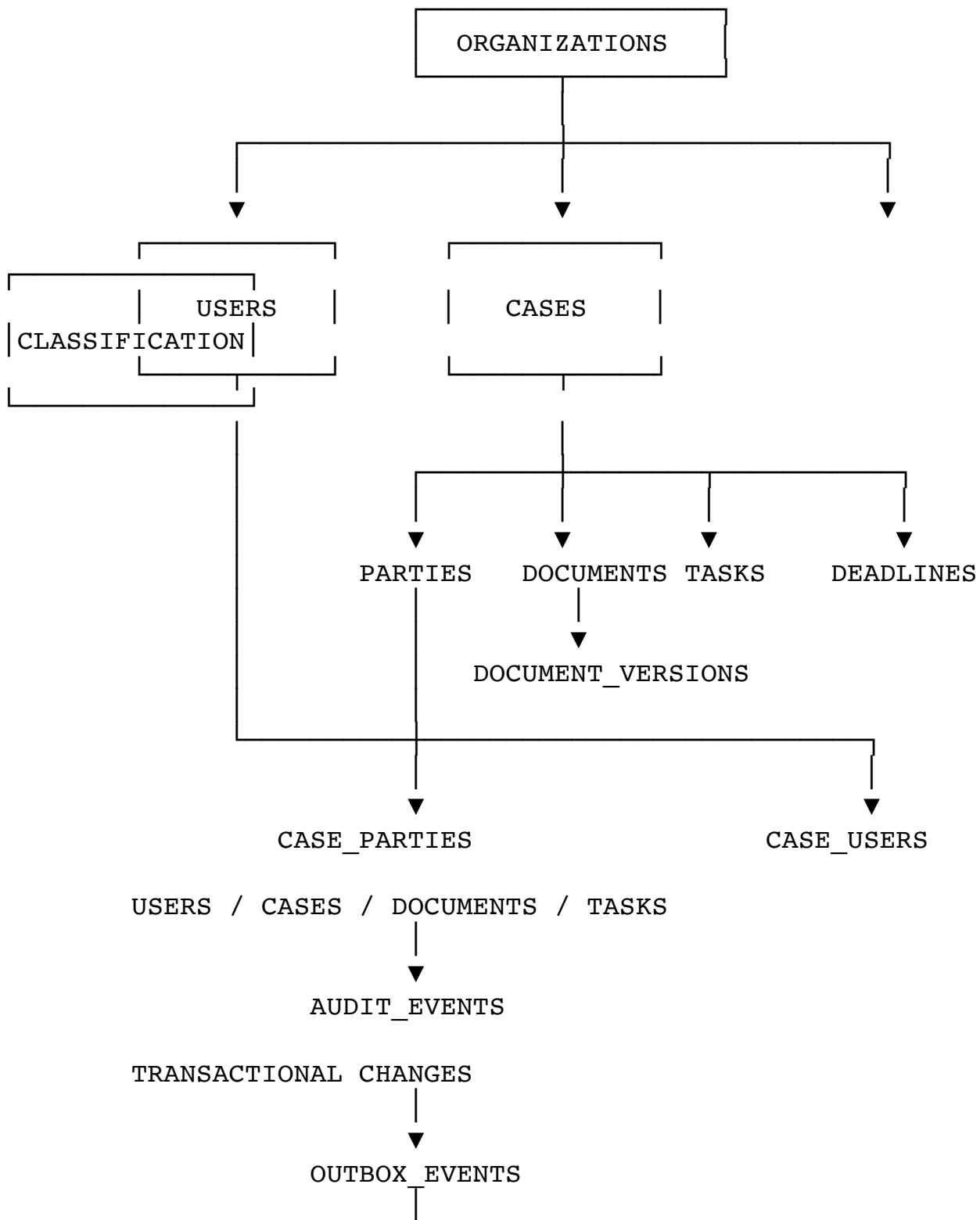
document type
document date
received date
classification
sender
recipient
case association
versioning

```
file integrity
```

searchable extracted text

Specific court filing requirements shall be implemented in the application/document workflow layer rather than assumed to apply to every document in the system.

104. ER Diagram — Logical View



105. Future Extensibility

The database should be capable of supporting future modules such as:

Billing
Time tracking
Invoices
Client portal
Court integration
Digital signatures
Advanced workflow
Knowledge management
AI-assisted document analysis
Document comparison
Legal research
Reporting

These should be added as bounded modules rather than substantially modifying the core case model.

106. Non-Goals

The database specification does not currently define:

- financial accounting
- billing calculations
- court electronic filing protocols
- digital-signature cryptographic implementation
- legal deadline calculation rules
- AI model storage
- external legal-research databases

Those should be specified separately when required.

107. Definition of Done — v1.3

The database implementation is complete when:

- PostgreSQL schema is implemented
- Alembic migrations are established
- core entities exist
- foreign keys are enforced
- uniqueness constraints are implemented
- indexes are implemented
- tenant relationships are enforced
- case/party relationships work
- document versioning works
- audit events can be persisted
- outbox events can be persisted
- migration tests pass
- rollback/recovery procedures are tested
- Icelandic UTF-8 data is tested
- security tests verify tenant isolation
- schema documentation is generated
- database deployment is integrated with CI/CD

108. Next Specification

The logical next step is:

Technical Specification v1.4 — REST API & OpenAPI Specification

v1.4 should translate the v1.2 backend architecture and v1.3 database model into the actual external API contract.

It should define:

- complete endpoint inventory
- HTTP methods
- URL structure

- request schemas
- response schemas
- pagination
- filtering
- sorting
- authentication
- authorization
- error codes
- validation rules
- case APIs
- party APIs
- document APIs
- document upload/download
- tasks
- deadlines
- search
- audit
- notifications
- asynchronous jobs
- idempotency
- API versioning
- OpenAPI examples
- frontend/backend contract
- API security requirements

This will give us a stable contract from which the FastAPI implementation can be built directly.

v1.3 now gives us the database foundation. The particularly important design decisions are the separation of **Case** → **Party relationships**, immutable **Document Versions**, tenant-aware records, and the **Audit + Transactional Outbox** combination. Those choices will make the subsequent API and application-service specifications much cleaner.

The natural next step is **v1.4 — REST API & OpenAPI Specification**, which can be detailed enough that we could subsequently start generating the actual FastAPI project from it.